

Instituto Tecnológico de Costa Rica

Escuela de Ingeniería en Computación

Curso

Compiladores e Intérpretes

Grupo

50

Asignación

Proyecto Compiladores - MiniGo

Docente(s)

Adriana Patricia Rojas Chavarria

Estudiante(s)

Rodríguez Díaz, Kenni Alexander

Ulate Camacho, Asdrubal

IS 2026

07 - 06 - 26

Tabla de Contenidos.

1. Introducción.
2. Análisis del problema.
3. Solución del problema.
4. Análisis de resultados.
5. Conclusiones.
6. Recomendaciones.
7. Bibliografía.
8. Anexos.

1. Introducción.

El proyecto Mini-Go Compilador e IDE representa una implementación completa y funcional de un compilador para subconjunto de reglas del lenguaje de programación creado por Google; “Go - Programming Language”.

Este proyecto representa una evaluación académica la cual surge como respuesta a la necesidad de desarrollar el conocimiento tanto lógico como de las herramientas utilizadas en la interpretación de lenguajes de programación, permitiendo así a los estudiantes comprender los procesos fundamentales de esta rama de la Ingeniería en Software.

El proyecto se divide en dos partes principalmente:

1.1. Compiler, cumple las siguientes responsabilidades:

- Análisis léxico.
- Análisis sintáctico.
- Análisis semántico.
- Generación de código.

1.2. IDE, cumple las siguientes responsabilidades:

- Editor de código simple.
- Mensajes de error detallados.
- “Terminal” integrada para visualizar resultados.
- Diseño moderno, simple e intuitivo para el usuario.

Este documento representa un análisis completo y detallado de la arquitectura, implementación, resultados obtenidos, conclusiones y recomendaciones derivadas de la realización de dicho compilador.

2. Análisis del problema.

2.1. General.

Si bien desarrollar o saber acerca de compiladores e intérpretes es una parte fundamental en la carrera de todo ingeniero, la realidad es que crear un compilador desde cero requiere de conocimientos extensos y avanzados, como; comprensión de los distintos tipos de lenguajes de programación, analizadores léxicos y sintácticos, tablas de símbolos y análisis semántico, entre otros.

El principal problema con esto es que todos estos conocimientos implementados de manera manual son, no solamente extensos de implementar, sino también propensos a errores y problemas por falta de retroalimentación visual, y al ser proyectos no solo extensos en cuanto a cantidad de código, sino también complejos en la implementación del mismo, se vuelve un proceso tedioso y poco optimizado para equipos pequeños.

2.2. Requerimientos.

- Crear un analizador léxico capaz de reconocer tokens.
- Crear un analizador sintáctico que genere un árbol.
- Crear un analizador semántico con manejo de scopes.
- Crear un IDE que proporcione retroalimentación visual.
- Generar código LLVM funcional.

2.3. Restricciones.

- Tabla de símbolos implementada en forma de lista secuencial.
- Magic numbers para identificación de tipos.
- Uso de ANTLR4 para el lexer y el parser.

3. Solución del problema.

Como se mencionó anteriormente, la solución se planteó a partir de dos grandes componentes: el backend o compilador, y el frontend o IDE.

Para el compilador se hizo uso del flujo básico visto en clase:

Lexer -> Parser -> AST -> Análisis Semántico -> Generación de código. -> LLVM IR -> Clang -> Ejecutable.

Luego de esto el IDE lee el ejecutable y proporciona los resultados si todo funciona correctamente, o proporciona los errores en caso de haber fallado.

3.1. Fases del backend.

3.1.1. Análisis léxico.

Se implementó mediante el uso de ANTLR, este analizador de grammar permite reconocer palabras reservadas, identificadores, literales, operadores, comentarios, delimitadores y errores léxicos.

3.1.2. Análisis sintáctico.

El parser, el cual también fue generado haciendo uso de ANTLR permite reconocer estructuras de declaración, validar instrucciones y expresiones, respetar la precedencia de los operadores, construir un árbol sintáctico, recuperar y reportar errores sintácticos, generar el árbol que sirve como entrada para el análisis semántico.

3.1.3. Análisis semántico.

Implementado haciendo uso del método visto en el curso mediante type_checker y symbol_table, permite mantener las tablas de tipos en

scopes anidados, validar los tipos de asignaciones y operaciones, comprobar variables en uso, detectar re-declaraciones, reportar errores con línea y columna, aislar errores

3.1.4. Generar código.

Mediante `encoder.go` cumple con generar código LLVM IR, manejar allocas, gestionar stack frames, generar instrucciones aritméticas, emitir el LLVM a `output.ll`, invocar a clang para hacer link al ejecutable.

3.2. Componentes del frontend.

3.2.1. Stack.

Se hizo principalmente uso de Electron + React + Typescript + Librerías de UI, como HeroUI para componentes, Tailwind.css para estilizado, Monaco Editor, Lucide React.

3.2.2. Flujo.

El flujo que sigue el IDE es el siguiente: el usuario ingresa código Mini-Go al editor -> cuando hacer clic en ejecutar se envía el contenido del archivo abierto al backend -> el compilador se encarga de procesar el código según las fases descritas anteriormente -> si falla: se reportan los errores en terminal, si es exitoso: se genera el ejecutable -> se muestra el output en la terminal integrada.

4. Análisis de resultados.

4.1. Cobertura funcional.

- Analizador léxico implementado.
- Analizador sintáctico implementado.
- Analizador semántico implementado.
- Generador de código implementado.
- Reporte de errores implementado.
- Manejo de archivos implementado.
- Terminal integrada implementada.
- IDE funcional implementado.

4.2. Rendimiento.

El compilador tuvo un rendimiento más que aceptable a lo largo de las pruebas realizadas, la compilación usando clang no toma recursos innecesarios y es prácticamente instantánea, el IDE es responsivo y tiene una alta capacidad de memoria caché.

Negativo podría decirse que el IDE a la hora de ejecutarlo tarda algunos segundos puesto que hace un build completo para evitar utilizar versiones anteriores de la compilación del compilador.

Positivo se puede destacar que la interacción entre compilador e IDE es prácticamente instantánea, tanto así que incluso se tuvo que agregar un delay al momento de leer la ejecución, puesto que trabajan en conjunto tan rápido que algunos discos duros antiguos no alcanzan a generar el output completo antes de que el IDE intente leerlo, así que para evitar este tipo de errores inesperados se agregaron unos cuantos milisegundos de retraso.

4.3. Casos de prueba.

Se hizo ejecución de múltiples casos de prueba, desde test simples para verificar el estado de una nueva funcionalidad, como por ejemplo la declaración de variables, hasta test complejos que ejemplifican en su totalidad la capacidad actual del compilador, a todos estos logró pasar sin problemas y con un tiempo de ejecución muy aceptable para no haberse tenido algún tipo de optimización en mente, en estos casos se evaluó lo siguiente:

- Variables simples y declaraciones múltiples.
- Operaciones aritméticas.
- Sentencias if-else.
- Bucles de tipo for.
- Funciones con parámetros.
- Retorno de valores.
- Prints.
- Errores detectados.
- Errores reportados.
- Variables no declaradas.
- Errores de syntaxis.
- If y for anidados.

5. Conclusiones.

Tras haber realizado la implementación del compilador para el subconjunto de Go; Mini-Go, se ha llegado a las siguientes conclusiones:

- Se logró desarrollar un compilador funcional que abarca todas las fases de la compilación tradicional.
- La integración del IDE con el compilador no fue tan complicada gracias a las herramientas modernas.
- El proyecto tiene mucho potencial de extensión, no solo porque ya existe un lenguaje padre al que mirar para optimizar su comportamiento, sino porque cuenta con lo justo necesario para seguir expandiéndose en una versión personalizada.

6. Recomendaciones.

A pesar de haber alcanzado un alto grado de satisfacción con la implementación del compilador, la realidad es que el actual compilador no es muy potente, y carece de muchas características que hacen destacar a Go como un buen lenguaje de programación, así como tener características limitadas en extensión comparadas al lenguaje original, como los structs, es por ello que se diseñó una lista de recomendaciones para futuras iteraciones del proyecto:

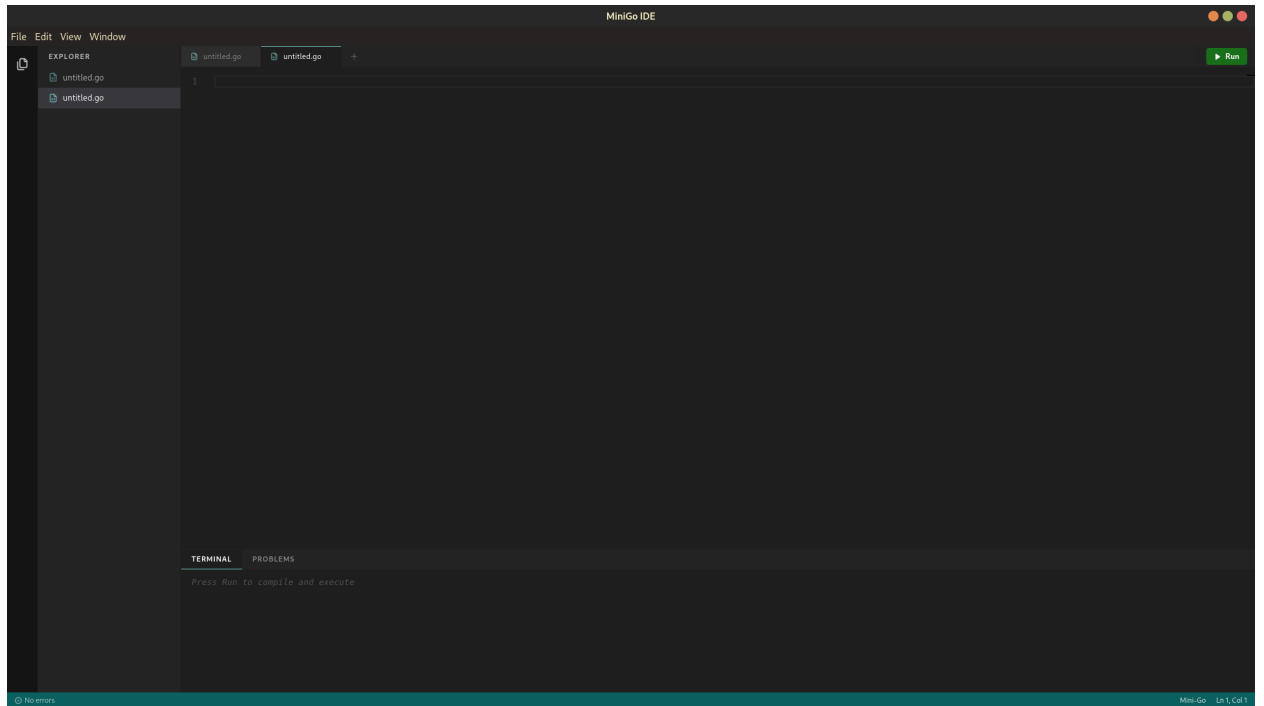
- Optimización de la tabla de símbolos.
- Expandir el soporte de tipos.
- Mejorar la recuperación de errores.
- Optimización general del código.
- Permitir el uso de paquetes externos.
- Interfaz para debug.
- Mejorar de UX; buscar en el archivo abierto o en todos los archivos.

7. Bibliografia.

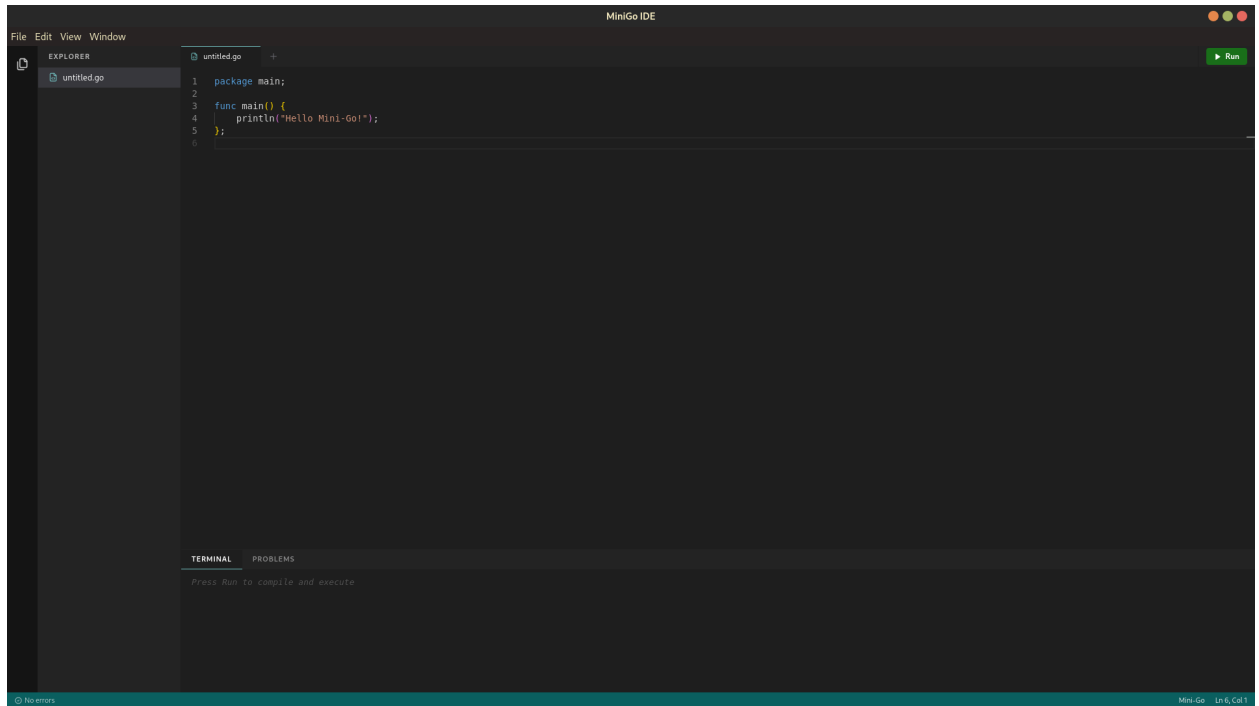
- **ANTLR Mega Tutorial** Tomassetti, F. (8 de marzo de 2017). *The ANTLR Mega Tutorial*. Tomassetti.me. <https://tomassetti.me/antlr-mega-tutorial/>
- **LLVM Tutorial** LLVM Project. (s. f.). *LLVM Tutorial: Table of Contents*. LLVM Reference Documentation. <https://llvm.org/docs/tutorial/>
- **Go Tutorial** The Go Authors. (s. f.). *Tutorial: Getting started with Go*. Go Documentation. <https://go.dev/doc/tutorial/>
- **React Documentation** Meta Platforms. (s. f.). *Quick Start*. React Documentation. <https://react.dev/learn>
- **Electron Tutorial** OpenJS Foundation. (s. f.). *Tutorial: Your First Application*. Electron. <https://www.electronjs.org/docs/latest/tutorial/tutorial-first-app>
- **Clang Getting Started** Clang Team. (s. f.). *Getting Started with the Clang System*. Clang C Language Family Frontend for LLVM. https://clang.llvm.org/get_started.html
- **GeeksforGeeks Compiler Design** GeeksforGeeks. (9 de mayo de 2026). *Compiler Design Tutorial*. GeeksforGeeks. <https://www.geeksforgeeks.org/compiler-design/compiler-design-tutorials/>

8. Anexos.

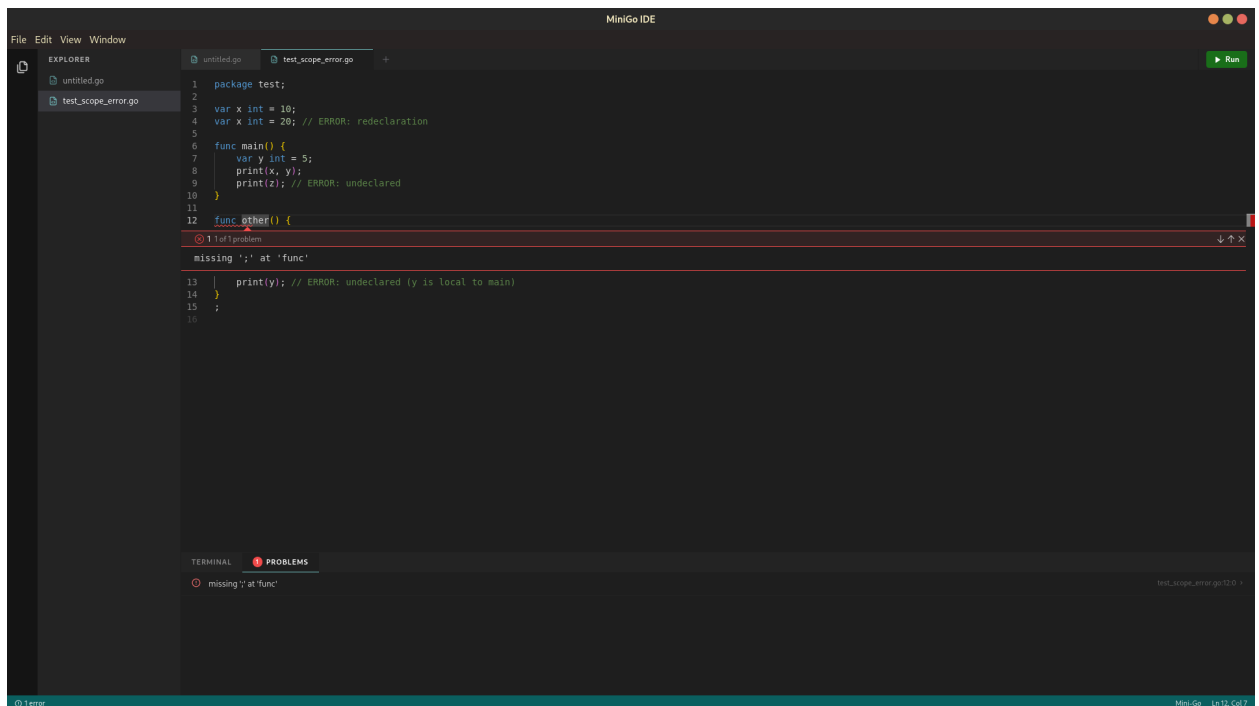
8.1. Pantalla de inicio.



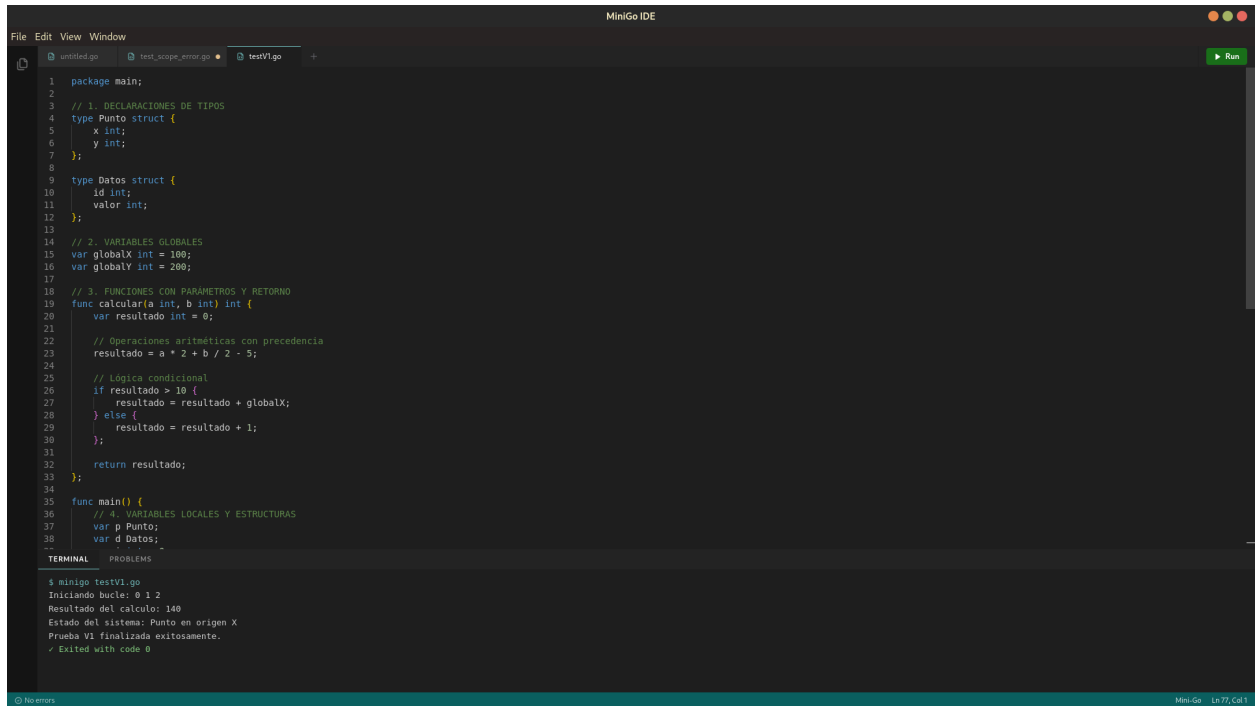
8.2. Editor de código.



8.3. Errores.



8.4. Ejecución.



The screenshot displays the MiniGo IDE interface. The editor window shows a Go program with the following code:

```
1 package main;
2
3 // 1. DECLARACIONES DE TIPOS
4 type Punto struct {
5     x int;
6     y int;
7 };
8
9 type Datos struct {
10     id int;
11     valor int;
12 };
13
14 // 2. VARIABLES GLOBALES
15 var globalX int = 100;
16 var globalY int = 200;
17
18 // 3. FUNCIONES CON PARAMETROS Y RETORNO
19 func calcular(a int, b int) int {
20     var resultado int = 0;
21
22     // Operaciones aritméticas con precedencia
23     resultado = a * 2 + b / 2 - 5;
24
25     // Lógica condicional
26     if resultado > 10 {
27         resultado = resultado + globalX;
28     } else {
29         resultado = resultado + 1;
30     };
31
32     return resultado;
33 };
34
35 func main() {
36     // 4. VARIABLES LOCALES Y ESTRUCTURAS
37     var p Punto;
38     var d Datos;
39     ...
40 }
```

The terminal window at the bottom shows the execution output:

```
$ minigo testV1.go
Iniciando bucle: 0 1 2
Resultado del calculo: 140
Estado del sistema: Punto en origen X
Prueba V1 finalizada exitosamente.
✓ Exited with code 0
```

8.5. Estructura.

```
├── backend
│   ├── parser
│   └── tests
├── docs
├── ide
│   ├── dist
│   │   └── assets
│   ├── dist-electron
│   ├── node_modules
│   │   ├── autoprefixer -> .pnpm/autoprefixer@10.5.0_postcss@8.5.15
│   │   │   /node_modules/autoprefixer
│   │   ├── electron -> .pnpm/electron@33.4.11/node_modules/electron
│   │   ├── framer-motion -> .pnpm/framer-motion@11.18.2_react-dom@18
│   │   │   .3.1_react@18.3.1_react@18.3.1/node_modules/framer-motion
│   │   ├── @heroui
│   │   │   └── react -> ../.pnpm/@heroui+react@2.8.10_@react
│   │   │       -spectrum+provider@3.11.1_react-dom@18.3.1_react@18.3
│   │   │       .1_rea_68289b720cee97f406a76c9aca9ff698/node_modules/@heroui/react
│   │   ├── lucide-react -> .pnpm/lucide-react@0.471.2_react@18.3.1
│   │   │   /node_modules/lucide-react
│   │   ├── @monaco-editor
│   │   │   └── react -> ../.pnpm/@monaco-editor+react@4.7.0_monaco
│   │   │       -editor@0.55.1_react-dom@18.3.1_react@18.3.1_react@18.3.1
│   │   │       /node_modules/@monaco-editor/react
│   │   ├── postcss -> .pnpm/postcss@8.5.15/node_modules/postcss
│   │   ├── react -> .pnpm/react@18.3.1/node_modules/react
│   │   ├── react-dom -> .pnpm/react-dom@18.3.1_react@18.3.1
│   │   │   /node_modules/react-dom
│   │   └── tailwindcss -> .pnpm/tailwindcss@3.4.19/node_modules
│   │       /tailwindcss
│   │   ├── @types
│   │   │   └── node -> ../.pnpm/@types+node@22.19.20/node_modules
│   │   │       /@types/node
│   │   ├── react -> ../.pnpm/@types+react@18.3.31/node_modules
│   │   │       /@types/react
│   │   ├── react-dom -> ../.pnpm/@types+react-dom@18.3.7_@types
│   │   │       +react@18.3.31/node_modules/@types/react-dom
│   │   ├── typescript -> .pnpm/typescript@5.9.3/node_modules
│   │   │       /typescript
│   │   ├── vite -> .pnpm/vite@6.4.3_@types+node@22.19.20_jiti@1.21.7
│   │   │       /node_modules/vite
│   │   ├── @vitejs
│   │   │   └── plugin-react -> ../.pnpm/@vitejs+plugin-react@4.7
│   │   │       .0_vite@6.4.3_@types+node@22.19.20_jiti@1.21.7_/node_modules/@vitejs
│   │   │       /plugin-react
│   │   ├── vite-plugin-electron -> .pnpm/vite-plugin-electron@0.28
│   │   │       .8_vite-plugin-electron-renderer@0.14.7/node_modules/vite-plugin
│   │   │       -electron
│   │   └── vite-plugin-electron-renderer -> .pnpm/vite-plugin
│   │       -electron-renderer@0.14.7/node_modules/vite-plugin-electron-renderer
│   └── src
```